

SnapSend: Phase-wise Build Plan

A hand-off document written so Claude (Claude Code) can pick up any phase and start working: project setup, the CLAUDE.md to paste into the repo, six phases with tasks, acceptance criteria, test checklists, and the exact prompt to start each phase with.

How to use this document: complete Phase 0 yourself (10 minutes), then open Claude Code in the repo and run the Phase 1 kickoff prompt. One phase at a time. Do not let any phase start before the previous phase's acceptance checklist passes.

Companion to: SnapSend Product Review & Business Plan (July 2026) · v1.1 — adds skill routing, code-review gate, and token-discipline rules (section 5)

1. How This Plan Works

The operating rules that keep a solo build on rails.

1.1 Phase map

Phase	Name	Est. time	Output you can touch
0	Project setup (human)	~30 min	Repo, CLAUDE.md, dev extension loading in Chrome
1	Capture core	Week 1	Floating bar + region snip + annotate + clipboard copy — usable daily from day 3
2	Send screen & WhatsApp draft	Week 2	Side panel, contacts, draft injected into WhatsApp Web with caption
3	Contacts & more channels	Week 3	Contact mirroring, send-by-number, Telegram draft, Gmail draft
4	Video	Week 4	Screen recording, mp4 conversion, video drafts, onboarding, fallback polish
5	Money & launch plumbing	Week 5	Backend, accounts, free-tier caps, Stripe/Razorpay, Web Store package
6	Beta & launch ops	Week 6+	Canary monitor, beta feedback loop, listing assets, launch checklist

1.2 Rules for working with Claude Code on this project

One phase per session-chain. Start each phase with its kickoff prompt (section 4). Within a phase, work task-by-task in order — each task below is sized to be one focused Claude Code instruction.

The acceptance checklist is the contract. A phase is done when every checklist item passes with you manually testing in Chrome — not when the code compiles. Paste failures back to Claude verbatim, including console errors and screenshots.

Commit per task, branch per phase. `phase-1-capture`, `phase-2-send`, etc. Merge to main only after the phase checklist passes. This keeps every working state recoverable.

Never let Claude expand scope. If a session suggests extra features, the answer is: log it in IDEAS.md and continue. Scope is decided in this document, not mid-session.

Selector fragility rule. Anything touching WhatsApp/Telegram DOM must read its selectors from src/config/selectors.json — never hard-coded inline. This single rule is what makes future breakage a 5-minute fix.

2. Phase 0 — Project Setup (you, not Claude)

Ten minutes of human work that every later session depends on.

2.1 Steps

1. Create a private GitHub repo, e.g. **snapsend-extension**. Clone locally.
2. Add the CLAUDE.md from section 2.2 to the repo root — this is the standing brief every Claude Code session reads automatically.
3. Create empty folders matching the file layout in 2.3.
4. In Chrome: `chrome://extensions` → enable Developer mode → you will use “Load unpacked” on the /dist folder from Phase 1 onward.
5. Optional but recommended: register the Chrome Web Store developer account now (\$5) so it is aged before launch.

2.2 CLAUDE.md — paste this into the repo root

```
# SnapSend — Chrome extension (Manifest V3)
## What this is
Capture (snip/record) anything in the browser, caption it, and place it as a
ready-to-send DRAFT in a chosen person's chat (WhatsApp Web / Telegram Web /
Gmail / Slack). The human always presses Send. We NEVER auto-send.

## Hard rules — never violate
- Manifest V3 only. No remote code (Chrome policy). Remote CONFIG json is ok.
- All contact data + media stay on-device (chrome.storage.local). Never add
network calls that transmit contacts or captures. Backend sees only
email/plan/usage-count.
- All WhatsApp/Telegram DOM selectors live in src/config/selectors.json.
Never hard-code a selector in logic files.
- Every injection path must have the clipboard fallback: image is ALREADY on
the clipboard before injection is attempted; on failure show toast
"Press Ctrl+V to paste".
- Minimal permissions. Justify any new permission in PERMISSIONS.md first.
- No new features beyond the current phase. Log ideas in IDEAS.md.

## Stack
Vanilla TS + Vite (CRXJS plugin) for the extension. No heavy UI framework —
side panel and overlays are plain TS + CSS. ffmpeg.wasm for video (phase 4).
Node/Express on Render + Stripe/Razorpay for billing (phase 5).

## Skills — use them, don't reinvent
At session start, list available skills. When a task matches a skill, USE it:
- UI/visual design work (side panel, floating bar, onboarding) → frontend-design
or equivalent installed UI skill. Follow its tokens; no default-looking UI.
- Motion/animation polish (bar collapse, toasts, snip overlay) → motion/animation
skill if installed; else CSS transforms+opacity only, 150-250ms, no JS anim libs.
- Marketing copy (store listing, landing, onboarding text) → marketing/copy skill
if installed; voice: plain, concrete, no hype words.
- Any other installed skill that matches a task (by its description) → use it.

## Code standards gate — every task ends with a review pass
Before declaring a task done, self-review against: TS strict passes; no any;
every async path has error handling that ends in a user-visible toast, never a
silent fail; no selector outside selectors.json; no unused permissions; no dead
code; functions under ~40 lines. Fix violations, then output a 5-line review
summary (what checked / what fixed). If a violation can't be fixed in-scope, say
so and stop.

## Token discipline — no loops, no waste
- TWO-STRIKE RULE: if the same error persists after 2 fix attempts, STOP.
Summarize what was tried and ask the human. Never attempt #3 of the same idea.
- Plan before code: for any task, a max-10-line plan first, then implement.
- Don't re-read files already in context; don't re-run builds without a change.
- Prefer targeted edits over rewriting files. Never regenerate a whole file to
change 3 lines.
- Terse output mode: no long explanations of code just written; report only
what changed, how to test, and open questions.
- One task per instruction. Finish, stop, wait for human verification.

## Testing
After changes: npm run build, then reload unpacked extension at /dist.
Manual test steps are listed per-phase in EXECUTION_PLAN (this PDF's repo copy).
```

2.3 Target file layout

```
snapsend-extension/  
■■ CLAUDE.md · PERMISSIONS.md · IDEAS.md  
■■ manifest.json (generated by Vite/CRXJS config)  
■■ src/  
■ ■■ background/ (service worker: commands, capture orchestration, tab mgmt)  
■ ■■ content/ (floating bar, snip overlay, per-site injectors)  
■ ■ ■■ injectors/whatsapp.ts · telegram.ts  
■ ■■ sidepanel/ (send screen UI: preview, annotate, caption, contacts)  
■ ■■ lib/ (clipboard.ts, storage.ts, annotate.ts, ffmpeg.ts)  
■ ■■ config/selectors.json  
■■ server/ (phase 5: express app for auth/billing)  
■■ dist/ (build output – Load unpacked points here)
```

3. The Phases in Detail

Each phase: goal, ordered tasks (one Claude Code instruction each), and the acceptance checklist that gates the next phase.

Phase 1 — Capture core (Week 1)

Goal: you personally use SnapSend for real client feedback by day 3, even though “send” is still just clipboard.

#	Task (one instruction to Claude Code)
1.1	Scaffold: Vite + CRXJS + TypeScript MV3 extension. Manifest with permissions: activeTab, scripting, storage, sidePanel, clipboardWrite, commands. Build must load unpacked cleanly.
1.2	Floating bar content script: small pill bottom-center with Snip and Record(disabled) buttons; draggable; collapses to a dot; per-site position + hidden state remembered in storage; excluded on chrome:// and Web Store pages.
1.3	Region snip: on Snip or Alt+Shift+S, dim the page with a crop overlay; on mouse-up, captureVisibleTab, crop to the region (canvas), open result in side panel.
1.4	Annotation mini-editor in the side panel: arrow, rectangle, text, blur (pixelate) tools + undo. Keep it one canvas component, no libraries.
1.5	Clipboard: on annotation done (or immediately after snip if untouched), write PNG to clipboard via navigator.clipboard.write; toast confirms 'Copied — paste anywhere'.
1.6	Caption field under the preview; 'Copy with caption' also puts the text on clipboard as a second step. Persist last 5 captures in storage.local (images as blobs in IndexedDB) with a small history strip.

Phase 1 acceptance: shortcut opens snip on any normal site · region capture is pixel-accurate on scrolled pages · all four annotation tools work with undo · pasted image lands correctly in WhatsApp Web, Gmail, Slack manually · bar never overlaps site chat widgets bottom-right · extension reload loses no saved positions. **You have used it for at least one real client task.**

Phase 2 — Send screen & WhatsApp draft (Week 2)

Goal: the magic moment exists — click a person, the draft with image + caption is sitting in their WhatsApp Web chat.

#	Task
2.1	Side panel send screen: capture preview on top, caption field, contacts grid below (pinned favorites row + searchable list). Contacts stored in chrome.storage.local as {name, phone, channel, pinned, lastUsed}.
2.2	Manual contact add: name + phone + channel picker. 'New number' quick-send field that skips saving.
2.3	WhatsApp open-to-chat: background worker finds an existing web.whatsapp.com tab or opens one, navigates to the direct-chat URL for the phone number, waits for the composer to exist (poll selectors from selectors.json with timeout).
2.4	Draft injection: dispatch a synthetic paste event carrying the image file into the composer; then set caption text. All selectors from selectors.json. On ANY failure at ANY step: focus composer if possible, toast 'Image copied — press Ctrl+V', never throw to the user.
2.5	selectors.json remote refresh: on service-worker start, fetch a raw GitHub URL of selectors.json (config only), cache with fallback to bundled copy. Version field + log line when remote differs.

#	Task
2.6	Mis-send guard: before injection, side panel shows 'Sending to {name} on WhatsApp' with avatar initial; a 700ms disabled state on the confirm button prevents double-click accidents.

Phase 2 acceptance: cold start (no WA tab) → draft appears within ~8s · warm (WA tab open) → under ~3s · caption lands under the image · fallback toast path verified by intentionally breaking a selector · nothing ever auto-sends (audit the code path) · works after Chrome restart.

Phase 3 — Contacts & more channels (Week 3)

Goal: pick anyone without opening WhatsApp; Telegram and Gmail work end-to-end.

#	Task
3.1	Contact mirroring: content script on web.whatsapp.com passively reads the visible recent-chats list (names + ids) when the tab is open, debounced, and merges into stored contacts. Read-only; rate-limited; selectors from selectors.json.
3.2	First-run sync flow: if contact store is empty, side panel shows 'Open WhatsApp Web once to sync' button that opens the tab and confirms when contacts arrive.
3.3	Telegram Web injector (src/content/injectors/telegram.ts): same open-to-chat + paste-injection + caption pattern; same fallback rule; selectors in the same json.
3.4	Gmail draft via API: OAuth (chrome.identity) with gmail.compose scope only; create a draft with the image attached and caption as body; open the draft in a tab. Document the scope in PERMISSIONS.md.
3.5	Per-contact channel memory: sending via a channel sets it as that contact's default; channel badge shown on every contact row; long-press/right-click to change.
3.6	History polish: reuse any of the last 5 captures with a new contact in two clicks.

Phase 3 acceptance: after one sync, sidebar lists your real recent WhatsApp chats searchable · brand-new phone number send works with zero setup · Telegram draft with caption verified · Gmail draft opens with attachment · channel memory survives restart · mirroring causes no visible jank in WhatsApp Web.

Phase 4 — Video (Week 4)

Goal: record → caption → video draft in chat; onboarding good enough for strangers.

#	Task
4.1	Recorder: Record button usesgetDisplayMedia (tab/window/screen via Chrome's picker) + mic toggle; MediaRecorder to webm; hard cap 3 minutes with a countdown pill; stop returns to side panel preview with a video player.
4.2	ffmpeg.wasm integration (lazy-loaded only when first needed): webm → mp4 (H.264/AAC), target under 16MB via CRF ladder; progress bar; conversion must not freeze the panel (worker).
4.3	Video draft injection for WhatsApp/Telegram: same synthetic-paste pattern with the mp4 file; fallback = mp4 saved to Downloads + toast explaining drag-into-chat.
4.4	Onboarding (first run): 3 steps — pin the extension → open WhatsApp Web once → send yourself a test snip. Progress checklist UI; under 90 seconds.
4.5	Quality pass: error toasts humanized, empty states, dark-mode-safe styles, 12 restricted-page edge cases (PDF viewer, Web Store, chrome://) degrade politely.

Phase 4 acceptance: 60s recording converts in under ~15s on a mid-range laptop · video draft plays in-chat after Send · 3-min cap enforced · onboarding tested on one friend's machine cold · no unhandled promise rejections in a full-day console log.

Phase 5 — Money & launch plumbing (Week 5)

Goal: accounts, caps, payments, and a Web-Store-ready package. Backend stays tiny.

#	Task
5.1	Server (in /server): Express on Render. Endpoints: magic-link email auth, GET /me (plan, usage), POST /usage/increment, Stripe + Razorpay webhooks. DB: one users table (email, plan, usage_month, created). Nothing else is stored — enforce in code review.
5.2	Extension auth: sign-in screen in side panel (email → magic link → token in storage). Anonymous use allowed with local-only counting until sign-in.
5.3	Free-tier caps: 15 sends/month, image-only, 3 saved contacts. Cap UI: friendly counter, upgrade sheet at the wall. Video + unlimited behind Pro.
5.4	Payments: Stripe Checkout (global) + Razorpay (India) links from the upgrade sheet; webhook flips plan; extension refetches /me. Annual plan included.
5.5	Store package: production build, listing copy, 5 screenshots + 1 short screencast, privacy policy page (local-first wording from the business plan), PERMISSIONS.md finalized, single-purpose statement. Submit for review.

Phase 5 acceptance: test-mode purchase upgrades a real account end-to-end · caps enforce and reset monthly · server audit: no endpoint can receive contacts or media · listing passes Chrome's review (allow up to a week; fix-and-resubmit is normal).

Phase 6 — Beta & launch ops (Week 6+)

#	Task
6.1	Canary: tiny scheduled GitHub Action that headlessly loads WhatsApp Web and checks selectors.json entries still match; alerts you (email/Telegram) on failure.
6.2	Feedback loop: in-panel 'Report issue' that pre-fills a GitHub issue (or simple form) with version + console tail (with user consent).
6.3	Beta: invite the 15–20 people listed in the business plan §10.2; one week of daily fixes; collect 5 quotable testimonials.
6.4	Launch assets: the 20-second demo clip in 4 variants; Product Hunt draft; 3 SEO pages ('send screenshot to WhatsApp from Chrome' etc.) on a simple landing site.
6.5	Launch per business plan §8.2. Post-launch: triage → selector patches → weekly release rhythm.

Phase 6 acceptance / go-live gate: canary green 7 days straight · at least 8 of 15 beta users still active in week 2 (if not — stop and diagnose retention before launching) · refund + support email flow tested · rollback plan written (previous version zip kept per release).

4. Kickoff Prompts for Each Phase

Copy-paste these to start each Claude Code session-chain. They assume CLAUDE.md is in the repo.

Phase 1 kickoff

Read CLAUDE.md fully. List your installed skills and tell me which ones you'll use for UI, motion, and code review in this project. Then execute Phase 1 (Capture core), task 1.1 only: scaffold the extension exactly as specified – Vite + CRXJS + TypeScript, MV3, the permission set listed, building to /dist so I can Load Unpacked. Set up the folder layout from the plan. Give a max-10-line plan first, implement, run the code standards gate, then stop and tell me how to load and verify. Do not start task 1.2 until I confirm.

Phase 2 kickoff

Read CLAUDE.md. Phase 1 is accepted and merged. We are starting Phase 2 (Send screen & WhatsApp draft). Before coding task 2.1, restate the hard rules that apply to this phase – especially the selectors.json rule, the clipboard-first fallback rule, and never-auto-send – then implement 2.1 only.

Phase 3–6 kickoffs

Same pattern: 'Read CLAUDE.md. Phase N-1 accepted. Starting Phase N. Restate applicable hard rules, then implement task N.1 only, stop for my verification.' – Keeping every session one-task-gated is what keeps quality high and scope flat.

When something breaks (especially WhatsApp updates)

WhatsApp Web injection stopped working today. Symptoms: [paste console errors + what you saw]. Open src/config/selectors.json and the whatsapp injector, inspect the current WhatsApp Web DOM with me, and produce ONLY an updated selectors.json – no logic changes unless selectors alone cannot fix it. Then tell me how to push the config so live users get it without a store review.

5. Skills, Review Gate & Token Discipline

How Claude works during this build: which skills to invoke, the quality gate every task passes, and the rules that prevent token-burning loops.

5.1 Skill routing — which skill for which work

At the start of every session, Claude Code must list its installed skills and use the ones that match the task. Skill names differ per machine — the table maps *work type* to *skill category*; Claude matches by each skill's own description, not by guessing names. If you have additional skills installed (add their names to CLAUDE.md's skills section once, e.g. your own custom ones), they get picked up automatically.

Work type	Skill category to invoke	Where it applies in this plan	If no such skill installed
UI / visual design	Frontend / UI design skill (e.g. frontend-design)	Side panel send screen (2.1), floating bar (1.2), annotation editor (1.4), onboarding (4.4), upgrade sheet (5.3)	Follow a written mini design system: one accent color, 8px spacing grid, system font stack, no gradients, no default-browser-looking buttons.
Motion design	Animation / motion skill if present	Bar collapse-to-dot, snip overlay dim-in, toast enter/exit, progress states (4.2)	CSS-only: transform+opacity, 150–250ms, ease-out; zero JS animation libraries (bundle size + MV3 review risk).
Marketing & copy	Marketing / copywriting skill if present	Store listing + screenshots copy (5.5), landing + SEO pages (6.4), Product Hunt draft, onboarding microcopy	Voice rules: plain words, concrete outcomes ('draft in their chat in 3 seconds'), zero hype adjectives, one idea per sentence.
Code review	Reviewer pass or code-review skill/subagent	END OF EVERY TASK, all phases — see 5.2	Run the 5.2 checklist as a self-review pass in the same session; a fresh-context review of the diff is better when available.
Docs & assets	PDF/docs skills as relevant	PERMISSIONS.md, privacy page, release notes	Plain markdown, kept short.

5.2 The code standards gate (runs at the end of every task)

A task is not 'done' until this review pass runs and reports. The gate checks:

#	Check	Fail example that must be caught
G1	TypeScript strict passes; no any; no ts-ignore without a comment justifying it	A quick 'as any' cast to silence the compiler.
G2	Every async/promise path has error handling ending in a user-visible toast — never a silent failure	Injection fails and nothing appears; user thinks the app is broken.
G3	No DOM selector exists outside src/config/selectors.json	A <code>querySelector('.copyable-area')</code> inline in <code>whatsapp.ts</code> .
G4	manifest permissions unchanged, or the change is justified in PERMISSIONS.md in the same commit	Adding 'tabs' permission 'just in case'.
G5	No dead code, no commented-out blocks, functions under ~40 lines, no new dependencies without approval	Pulling in lodash for one debounce.

#	Check	Fail example that must be caught
G6	The clipboard fallback path still works (phases 2+): image is on clipboard BEFORE injection is attempted	A refactor that moves clipboard-write to after injection success.
G7	Output: a 5-line review summary — what was checked, what was fixed, what (if anything) is deferred	A wall of self-congratulation instead of a report.

5.3 Token discipline — the anti-loop rules

These rules exist because agentic sessions burn most of their tokens in two ways: retry loops on a stubborn bug, and re-narrating/re-reading things already in context. Both are banned:

Rule	What it means in practice
Two-strike rule	If the same error survives two distinct fix attempts, Claude STOPS — summarizes the error, both attempts, and its best hypothesis in under 10 lines, then waits for the human. A third attempt at the same idea is never allowed. (Humans debug stubborn extension issues faster with a screenshot than Claude does blind.)
Plan-then-code	Every task starts with a max-10-line plan (files touched, approach, risks). No code before the plan. This catches wrong directions at 100 tokens instead of 10,000.
No redundant reads/builds	Never re-read a file already in context unless it was edited since; never re-run a build without a code change; never 'verify' by regenerating what already exists.
Surgical edits	Change 3 lines by editing 3 lines. Whole-file regeneration is allowed only for new files.
Terse reporting	After a task: what changed (bullet paths), how to test (numbered steps), open questions. No prose explaining code line-by-line, no restating the task back.
One task, then stop	Sessions never continue to the next numbered task without explicit human 'go'. This is the single biggest loop-preventer: the human verification between tasks is where drift gets caught cheap.
Context resets per phase	Start each phase with a fresh session (the kickoff prompts in section 4). Long sessions accumulate stale context that causes exactly the confusion loops we're avoiding.

Why this section matters more than it looks: on a 6-week agentic build, undisciplined sessions can spend 30–50% of tokens on retry loops and re-narration. These rules are already embedded in the CLAUDE.md (section 2.2), so every session inherits them automatically — this section is your reference for enforcing them when a session drifts. The enforcement phrase is simply: *'Two-strike rule — stop and summarize.'*

6. Standing Decision Log

Decisions already made — Claude sessions should not reopen these:

Decision	Ruling	Why
UI framework	None (vanilla TS + CSS)	Extension surfaces are small; frameworks bloat MV3 bundles and review risk.
Where media lives	Device only, ever	The privacy promise is the product's trust story and legal shield.
Auto-send	Never, no setting for it	ToS survival line + mis-send safety; human presses Send, always.

Decision	Ruling	Why
Browsers	Chrome only in v1	Firefox/Safari are IDEAS.md items; ports before revenue are waste.
Desktop capture	Chrome's picker is enough for v1	Native helper app is the v2 gate at Base revenue scenario.
Selector storage	selectors.json + remote config	Turns recurring WhatsApp breakage into config pushes, not releases.
Pricing	Free 15 sends / Pro ~\$6 or ~\$299	Set in the business plan; revisit only after 100 paying users.

Final note to the builder: the plan's only non-negotiable sequencing is Phase 1 before everything — because the moment you are using it daily on real client work, every later decision gets made by evidence instead of guesswork. Start there, and start this week.